CS24304 • PRACTICAL LAB MANUAL

Data Mining Concepts & Techniques Lab Manual

Complete 10 Practical Experiments with Python, Apriori, FP-Growth, PCA, k-Means & Naive Bayes

Author: Shaswat Raj (BIT Mesra CSE) **Academic Scheme:** NEP 2024–25 (5th Sem) **Credits:** 1.5 Practical Credits

Experiment 1: Statistical Data Description & Proximity Measures in Python

Objective: Compute five-number summaries (Min, Q_1, Median, Q_3, Max), IQR outlier fences, and calculate Euclidean, Manhattan, Minkowski, Cosine, and Jaccard similarity metrics in Python.

```
import numpy as np

def five_number_summary(data):
    q1 = np.percentile(data, 25)
    median = np.median(data)
    q3 = np.percentile(data, 75)
    iqr = q3 - q1
    lower_fence = q1 - 1.5 * iqr
    upper_fence = q3 + 1.5 * iqr
    outliers = [x for x in data if x < lower_fence or x > upper_fence]
    return {'Min': np.min(data), 'Q1': q1, 'Median': median, 'Q3': q3, 'Max': np.max(data),
            'IQR': iqr, 'Outliers': outliers}

def proximity_measures(u, v):
    u, v = np.array(u), np.array(v)
    euclidean = np.linalg.norm(u - v)
    manhattan = np.sum(np.abs(u - v))
    cosine_sim = np.dot(u, v) / (np.linalg.norm(u) * np.linalg.norm(v))
    # Jaccard for binary vectors
    jaccard = np.sum(np.bitwise_and(u > 0, v > 0)) / np.sum(np.bitwise_or(u > 0, v > 0))
    return {'Euclidean': euclidean, 'Manhattan': manhattan, 'Cosine': cosine_sim, 'Jaccard': jaccard}

data = [12, 14, 15, 18, 19, 21, 22, 24, 25, 28, 95] # 95 is outlier
print("Five-Number Summary:", five_number_summary(data))
print("Proximity Metrics:", proximity_measures([1, 1, 0, 1], [1, 0, 0, 1]))
```

Experiment 2: Data Preprocessing: Binning Smoothing & Normalization

Objective: Implement Equi-width and Equi-depth data binning (by bin means/boundaries), Min-Max normalization, and Z-score standardization from scratch.

```
def bin_by_means(data, num_bins=3):
    sorted_data = sorted(data)
    bin_size = len(data) // num_bins
    smoothed = []
    for i in range(0, len(data), bin_size):
        curr_bin = sorted_data[i:i + bin_size]
        mean_val = round(sum(curr_bin) / len(curr_bin), 2)
        smoothed.extend([mean_val] * len(curr_bin))
    return smoothed

def min_max_normalize(data, new_min=0.0, new_max=1.0):
    min_v, max_v = min(data), max(data)
    return [((x - min_v) / (max_v - min_v)) * (new_max - new_min) + new_min for x in data]

def z_score_standardize(data):
    mean = sum(data) / len(data)
    std = (sum((x - mean) ** 2 for x in data) / len(data)) ** 0.5
    return [(x - mean) / std for x in data]

raw_salaries = [200, 300, 400, 600, 1000, 1500, 2100, 2900, 4600]
print("Smoothed by Bin Means:", bin_by_means(raw_salaries, 3))
print("Min-Max [0, 1]:       ", [round(x, 3) for x in min_max_normalize(raw_salaries)])
print("Z-Score Standardized: ", [round(x, 3) for x in z_score_standardize(raw_salaries)])
```

Experiment 3: Correlation & Chi-Square (χ^2) Independence Test in Python

Objective: Calculate Pearson Correlation coefficient r for numeric attributes and perform the χ^2 hypothesis test of independence for 2×2 contingency tables.

```
import numpy as np

def chi_square_test(contingency_table):
    # Observed table: 2D array
    O = np.array(contingency_table)
    row_sums = O.sum(axis=1, keepdims=True)
    col_sums = O.sum(axis=0, keepdims=True)
    total = O.sum()
    E = (row_sums @ col_sums) / total # Expected frequencies

    chi2 = np.sum((O - E) ** 2 / E)
    dof = (O.shape[0] - 1) * (O.shape[1] - 1)
    return chi2, dof

# Table: Gender vs Course Preference [CSE, ECE]
# [ [Male_CSE, Male_ECE], [Female_CSE, Female_ECE] ]
table = [[250, 200], [50, 1000]]
chi2, dof = chi_square_test(table)
print(f"Chi-Square Statistic: {chi2:.4f} | Degrees of Freedom: {dof}")
```

Experiment 4: Principal Component Analysis (PCA) via Eigen-Decomposition

Objective: Implement PCA dimensionality reduction in pure NumPy: mean centering, covariance matrix computation, eigen-decomposition, and projection onto top-k eigenvectors.

```
import numpy as np

def pca_from_scratch(X, k=2):
    # 1. Zero-center data
    X_centered = X - np.mean(X, axis=0)
    # 2. Covariance Matrix
    cov_matrix = np.cov(X_centered, rowvar=False)
    # 3. Eigenvalue Decomposition
    eigenvalues, eigenvectors = np.linalg.eigh(cov_matrix)
    # 4. Sort in descending order
    idx = np.argsort(eigenvalues)[::-1]
    eigenvalues = eigenvalues[idx]
    eigenvectors = eigenvectors[:, idx]
    # 5. Project data onto top k eigenvectors
    top_k_components = eigenvectors[:, :k]
    X_reduced = np.dot(X_centered, top_k_components)
    explained_variance_ratio = eigenvalues[:k] / np.sum(eigenvalues)
    return X_reduced, explained_variance_ratio

X = np.array([[2.5, 2.4], [0.5, 0.7], [2.2, 2.9], [1.9, 2.2], [3.1, 3.0], [2.3, 2.7]])
X_proj, var_ratio = pca_from_scratch(X, k=1)
print("Projected 1D Coordinates:\n", X_proj)
print("Explained Variance Ratio:", var_ratio)
```

Experiment 5: Association Rule Mining: Complete Apriori Algorithm

Objective: Implement the Apriori algorithm from scratch in Python: frequent 1-itemsets generation, candidate generation ($L_{k-1} \bowtie L_{k-1}$), subset pruning, and confidence-based rule generation.

```
from itertools import combinations

def get_support(itemset, transactions):
    count = sum(1 for t in transactions if set(itemset).issubset(t))
    return count / len(transactions)

def apriori(transactions, min_support=0.5, min_confidence=0.7):
    # 1. Generate 1-itemset candidates
    items = sorted(list(set(i for t in transactions for i in t)))
    L1 = {frozenset([i]): get_support([i], transactions)
          for i in items if get_support([i], transactions) ≥ min_support}

    current_L = L1
    all_frequent = dict(L1)
    k = 2

    while current_L:
        # Candidate Generation
        prev_itemsets = list(current_L.keys())
        candidates = set()
        for i in range(len(prev_itemsets)):
            for j in range(i + 1, len(prev_itemsets)):
                union = prev_itemsets[i] | prev_itemsets[j]
                if len(union) == k:
                    # Pruning step: all (k-1) subsets must be frequent
                    if all(frozenset(s) in current_L for s in combinations(union, k - 1)):
                        candidates.add(union)

        # Filter by min_support
        next_L = {c: get_support(c, transactions)
                  for c in candidates if get_support(c, transactions) ≥ min_support}
        all_frequent.update(next_L)
        current_L = next_L
        k += 1

    # Rule Generation
    rules = []
    for itemset, supp in all_frequent.items():
        if len(itemset) ≥ 2:
            for i in range(1, len(itemset)):
                for antecedent in combinations(itemset, i):
                    antecedent = frozenset(antecedent)
                    consequent = itemset - antecedent
                    conf = supp / all_frequent[antecedent]
                    if conf ≥ min_confidence:
                        rules.append((antecedent, consequent, supp, conf))
    return all_frequent, rules

transactions = [
    {'Bread', 'Milk'},
    {'Bread', 'Diaper', 'Beer', 'Eggs'},
    {'Milk', 'Diaper', 'Beer', 'Cola'},
    {'Bread', 'Milk', 'Diaper', 'Beer'},
    {'Bread', 'Milk', 'Diaper', 'Cola'}
]

freq, rules = apriori(transactions, min_support=0.6, min_confidence=0.7)
print(f"Frequent Itemsets: {len(freq)}")
for ant, con, supp, conf in rules:
    print(f"Rule: {set(ant)} → {set(con)} | Supp={supp:.2f}, Conf={conf:.2f}")
```

Experiment 6: FP-Tree Construction & FP-Growth Algorithm

Objective: Construct an FP-Tree data structure with header tables and node links, extracting frequent patterns without costly candidate generation.

```
class FPNode:
    def __init__(self, item, count, parent):
        self.item = item
        self.count = count
        self.parent = parent
        self.children = {}
        self.next_similar = None

class FPTree:
    def __init__(self):
        self.root = FPNode(None, 1, None)
        self.header_table = {}

    def insert(self, transaction, count=1):
        curr = self.root
        for item in transaction:
            if item in curr.children:
                curr.children[item].count += count
            else:
                new_node = FPNode(item, count, curr)
                curr.children[item] = new_node
                # Update header table links
                if item in self.header_table:
                    new_node.next_similar = self.header_table[item]
                    self.header_table[item] = new_node
                else:
                    self.header_table[item] = new_node
            curr = curr.children[item]
```

Experiment 7: k-Means Clustering from Scratch in Python

Objective: Implement Lloyd's k-Means clustering algorithm with centroid initialization, cluster assignment via Euclidean distance, centroid update, and Within-Cluster Sum of Squares (WCSS) evaluation.

```
import numpy as np

class KMeansScratch:
    def __init__(self, k=3, max_iters=100):
        self.k = k
        self.max_iters = max_iters

    def fit(self, X):
        np.random.seed(42)
        self.centroids = X[np.random.choice(X.shape[0], self.k, replace=False)]
        for _ in range(self.max_iters):
            # 1. Assign clusters
            distances = np.linalg.norm(X[:, np.newaxis] - self.centroids, axis=2)
            self.labels = np.argmin(distances, axis=1)
            # 2. Recompute centroids
            new_centroids = np.array([X[self.labels == i].mean(axis=0) for i in range(self.k)])
            if np.allclose(self.centroids, new_centroids): break
            self.centroids = new_centroids
        self.wcss = sum(np.sum((X[self.labels == i] - self.centroids[i]) ** 2) for i in range(self.k))

X = np.array([[1, 2], [1, 4], [1, 0], [10, 2], [10, 4], [10, 0]])
kmeans = KMeansScratch(k=2)
kmeans.fit(X)
print("Converged Centroids:\n", kmeans.centroids)
print("Cluster Assignments:", kmeans.labels)
print("WCSS Inertia:       ", kmeans.wcss)
```

Experiment 8: Density-Based Clustering: DBSCAN from Scratch

Objective: Implement DBSCAN (Density-Based Spatial Clustering of Applications with Noise) in Python to discover non-spherical clusters and filter noise anomalies (ϵ, MinPts).

```
import numpy as np

def dbscan_from_scratch(X, eps=1.5, min_pts=2):
    labels = [0] * len(X) # 0: unvisited, -1: noise, >0: cluster_id
    cluster_id = 0

    for i in range(len(X)):
        if labels[i] ≠ 0: continue
        # Find neighbors within eps
        neighbors = [j for j in range(len(X)) if np.linalg.norm(X[i] - X[j]) ⩽ eps]
        if len(neighbors) < min_pts:
            labels[i] = -1 # Noise
        else:
            cluster_id += 1
            labels[i] = cluster_id
            queue = list(neighbors)
            while queue:
                pt = queue.pop(0)
                if labels[pt] == -1: labels[pt] = cluster_id
                if labels[pt] == 0:
                    labels[pt] = cluster_id
                    pt_neighbors = [k for k in range(len(X)) if np.linalg.norm(X[pt] - X[k]) ⩽ eps]
                    if len(pt_neighbors) ⩾ min_pts:
                        queue.extend(pt_neighbors)
    return labels
```

Experiment 9: Naïve Bayes Classifier with Laplace Smoothing

Objective: Implement Gaussian/Multinomial Naïve Bayes Classifier in Python, estimating prior and conditional likelihood probabilities with Laplace add-1 smoothing.

```
import numpy as np

class NaiveBayesClassifier:
    def fit(self, X, y):
        self.classes = np.unique(y)
        self.mean = {}
        self.var = {}
        self.priors = {}
        for c in self.classes:
            X_c = X[y == c]
            self.mean[c] = np.mean(X_c, axis=0)
            self.var[c] = np.var(X_c, axis=0) + 1e-6
            self.priors[c] = len(X_c) / len(X)

    def _pdf(self, class_idx, x):
        mean = self.mean[class_idx]
        var = self.var[class_idx]
        numerator = np.exp(-((x - mean) ** 2) / (2 * var))
        denominator = np.sqrt(2 * np.pi * var)
        return numerator / denominator

    def predict(self, X):
        predictions = []
        for x in X:
            posteriors = []
            for c in self.classes:
                prior = np.log(self.priors[c])
                conditional = np.sum(np.log(self._pdf(c, x)))
                posteriors.append(prior + conditional)
            predictions.append(self.classes[np.argmax(posteriors)])
        return np.array(predictions)
```

Experiment 10: Classification Model Evaluation: Confusion Matrix & ROC-AUC

Objective: Compute Precision, Recall, F_1-Score, Specificity, and False Positive Rate (FPR) from a test confusion matrix, plotting ROC curves.

```
def evaluate_classifier(tp, fn, fp, tn):
    accuracy = (tp + tn) / (tp + fn + fp + tn)
    precision = tp / (tp + fp) if (tp + fp) > 0 else 0
    recall = tp / (tp + fn) if (tp + fn) > 0 else 0
    f1 = 2 * (precision * recall) / (precision + recall) if (precision + recall) > 0 else 0
    specificity = tn / (tn + fp) if (tn + fp) > 0 else 0
    return {'Accuracy': accuracy, 'Precision': precision, 'Recall': recall, 'F1-Score': f1, 'Specificity': specificity}

metrics = evaluate_classifier(tp=85, fn=15, fp=10, tn=190)
for k, v in metrics.items():
    print(f"{k:12}: {v*100:.2f}%")
```

Experiment 11: Hierarchical Agglomerative Clustering (AGNES) in Python

Objective: Implement bottom-up Hierarchical Agglomerative Clustering with Single Linkage ($d_{\min}$), Complete Linkage ($d_{\max}$), and Average Linkage (d_{avg}) to generate proximity dendrogram matrices.

```
import numpy as np

def single_linkage_distance(cluster_a, cluster_b, data):
    return min(np.linalg.norm(data[i] - data[j]) for i in cluster_a for j in cluster_b)

def agnes_clustering(data, target_k=2):
    clusters = [[i] for i in range(len(data))]
    while len(clusters) > target_k:
        min_dist = float('inf')
        merge_pair = (0, 1)
        for i in range(len(clusters)):
            for j in range(i + 1, len(clusters)):
                d = single_linkage_distance(clusters[i], clusters[j], data)
                if d < min_dist:
                    min_dist = d
                    merge_pair = (i, j)
        i, j = merge_pair
        clusters[i].extend(clusters[j])
        clusters.pop(j)
    return clusters

points = np.array([[1, 2], [2, 3], [8, 7], [9, 8], [1, 3]])
print("AGNES 2-Cluster Result:", agnes_clustering(points, target_k=2))
```

Experiment 12: Multidimensional OLAP Cube Operations in Python

Objective: Simulate a 3D Multidimensional Data Cube (Time $\times$ Item $\times$ Location $\rightarrow$ Sales) and execute Roll-Up, Drill-Down, Slice, Dice, and Pivot operations using NumPy/Pandas.

```
import numpy as np

# 3D Data Cube: Dimensions = [Quarter(4), Location(3), Product(2)]
np.random.seed(42)
cube = np.random.randint(100, 500, size=(4, 3, 2))

# 1. Roll-Up: Sum across Product dimension → 2D Matrix (Quarter x Location)
rollup_sales = np.sum(cube, axis=2)
# 2. Slice: Fix Quarter = Q1 (index 0) → 2D Matrix (Location x Product)
slice_q1 = cube[0, :, :]
# 3. Dice: Sub-cube for Q1..Q2 and Location 0..1
dice_subcube = cube[0:2, 0:2, :]
# 4. Pivot: Transpose Location and Product axes for 2D view
pivot_view = np.transpose(slice_q1)

print("Roll-Up Total Sales (Quarter x Location):\n", rollup_sales)
print("Slice for Q1 (Location x Product):\n", slice_q1)
print("Pivoted Q1 (Product x Location):\n", pivot_view)
```

Experiment 13: Linear Regression & Gradient Descent in Python

Objective: Implement Ordinary Least Squares (OLS) closed-form formula and Batch Gradient Descent from scratch to fit a linear predictive model ($y = wx + b$).

```
import numpy as np

def linear_regression_gd(X, y, lr=0.01, epochs=1000):
    w, b = 0.0, 0.0
    n = len(X)
    for _ in range(epochs):
        y_pred = w * X + b
        dw = -(2 / n) * np.sum(X * (y - y_pred))
        db = -(2 / n) * np.sum(y - y_pred)
        w -= lr * dw
        b -= lr * db
    return w, b

X = np.array([1, 2, 3, 4, 5], dtype=float)
y = np.array([2.2, 3.9, 6.1, 7.8, 10.2], dtype=float)
w, b = linear_regression_gd(X, y)
print(f"Fitted Model: y = {w:.4f} * x + {b:.4f}")
```

Experiment 14: Decision Tree Classifier via Gini Impurity (CART Algorithm)

Objective: Implement the CART (Classification and Regression Trees) algorithm from scratch in Python using binary splitting based on minimum Gini Impurity.

```
import numpy as np

def gini_impurity(y):
    _, counts = np.unique(y, return_counts=True)
    probs = counts / len(y)
    return 1.0 - np.sum(probs ** 2)

def best_split(X, y):
    best_gini = float('inf')
    best_feat, best_thresh = None, None
    n_samples, n_feats = X.shape
    for feat in range(n_feats):
        thresholds = np.unique(X[:, feat])
        for thresh in thresholds:
            left_mask = X[:, feat] <= thresh
            right_mask = ~left_mask
            if np.sum(left_mask) == 0 or np.sum(right_mask) == 0: continue
            gini_left = gini_impurity(y[left_mask])
            gini_right = gini_impurity(y[right_mask])
            weighted_gini = (np.sum(left_mask)/n_samples)*gini_left + (np.sum(right_mask)/n_samples)*gini_right
            if weighted_gini < best_gini:
                best_gini = weighted_gini
                best_feat, best_thresh = feat, thresh
    return best_feat, best_thresh, best_gini

X = np.array([[2.7, 2.5], [1.4, 2.3], [3.3, 4.4], [1.3, 1.8], [3.0, 3.0], [7.6, 2.7], [5.3, 2.0], [6.9, 1.7]])
y = np.array([0, 0, 0, 0, 0, 1, 1, 1])
feat, thresh, gini = best_split(X, y)
print(f"Optimal CART Split: Feature {feat} <= {thresh:.2f} (Gini = {gini:.4f})")
```

Experiment 15: Support Vector Machine (SVM) Linear Classifier in Python

Objective: Implement a linear Support Vector Machine (SVM) using Hinge Loss and Stochastic Gradient Descent (SGD) to find the maximum-margin hyperplane ($w \cdot x - b = 0$).

```
import numpy as np

class LinearSVM:
    def __init__(self, lr=0.001, lambda_param=0.01, n_iters=1000):
        self.lr = lr
        self.lambda_param = lambda_param
        self.n_iters = n_iters
        self.w = None
        self.b = None

    def fit(self, X, y):
        # Labels should be -1 and 1
        y_ = np.where(y <= 0, -1, 1)
        n_samples, n_features = X.shape
        self.w = np.zeros(n_features)
        self.b = 0

        for _ in range(self.n_iters):
            for idx, x_i in enumerate(X):
                condition = y_[idx] * (np.dot(x_i, self.w) - self.b) >= 1
                if condition:
                    self.w -= self.lr * (2 * self.lambda_param * self.w)
                else:
                    self.w -= self.lr * (2 * self.lambda_param * self.w - np.dot(x_i, y_[idx]))
                    self.b -= self.lr * y_[idx]

    def predict(self, X):
        approx = np.dot(X, self.w) - self.b
        return np.sign(approx)

svm = LinearSVM()
svm.fit(X, y)
print("Fitted SVM Hyperplane Weights:", svm.w, "| Bias:", svm.b)
```

Experiment 16: Ensemble Learning: Random Forest & AdaBoost in Python

Objective: Implement an ensemble of decision stumps using AdaBoost sequential re-weighting ($\alpha_t = \frac{1}{2}\ln\left(\frac{1-\epsilon_t}{\epsilon_t}\right)$) to convert weak classifiers into a robust strong classifier.

```
import numpy as np

class DecisionStump:
    def __init__(self):
        self.polarity = 1
        self.feature_idx = None
        self.threshold = None
        self.alpha = None

    def predict(self, X):
        n_samples = X.shape[0]
        X_column = X[:, self.feature_idx]
        predictions = np.ones(n_samples)
        if self.polarity == 1:
            predictions[X_column < self.threshold] = -1
        else:
            predictions[X_column > self.threshold] = -1
        return predictions

def adaboost_train(X, y, n_clf=5):
    n_samples, n_features = X.shape
    w = np.full(n_samples, (1 / n_samples))
    clfs = []
    for _ in range(n_clf):
        clf = DecisionStump()
        min_error = float('inf')
        for feat_i in range(n_features):
            X_column = X[:, feat_i]
            thresholds = np.unique(X_column)
            for threshold in thresholds:
                for polarity in [1, -1]:
                    predictions = np.ones(n_samples)
                    if polarity == 1: predictions[X_column < threshold] = -1
                    else: predictions[X_column > threshold] = -1
                    error = sum(w[y != predictions])
                    if error < min_error:
                        min_error = error
                        clf.polarity = polarity
                        clf.threshold = threshold
                        clf.feature_idx = feat_i
        EPS = 1e-10
        clf.alpha = 0.5 * np.log((1.0 - min_error + EPS) / (min_error + EPS))
        predictions = clf.predict(X)
        w *= np.exp(-clf.alpha * y * predictions)
        w /= np.sum(w)
        clfs.append(clf)
    return clfs
```

Comprehensive Viva-Voce Question Bank & Model Answers

Q1. What is the Apriori Property and why is it crucial for frequent pattern mining?

Apriori Property (Anti-monotonicity of support): All non-empty subsets of a frequent itemset must also be frequent. If an itemset I is infrequent ($Support(I) < \text{min\_sup}$), then any superset $I \cup \{A\}$ will also be infrequent and can be immediately pruned without scanning the transactional database!

Q2. How does FP-Growth achieve massive speedups over Apriori?

FP-Growth scans the database only twice to build a compact, highly compressed **FP-Tree** (Frequent Pattern Tree) in memory. It then mines frequent patterns via recursive conditional pattern bases without generating explosive candidate itemsets ($O(2^d)$ candidate generation elimination).

Q3. What is the difference between k-Means and DBSCAN Clustering?

- **k-Means:** Partition-based, requires predefined k, assumes spherical clusters, highly sensitive to outliers.
- **DBSCAN:** Density-based (ϵ, MinPts), automatically determines number of clusters, discovers arbitrary complex non-spherical shapes, and explicitly labels noise/outliers as -1.

Q4. Explain the Naïve Bayes Conditional Independence Assumption.

It assumes that the occurrence of each feature is strictly conditionally independent of all other features given the class label: $P(x_1, x_2, \ldots, x_n \mid C_k) = \prod_{i=1}^{n} P(x_i \mid C_k)$. Even when violated in practice, it yields stellar classification accuracy!

Q5. Explain the difference between Star Schema and Snowflake Schema in Data Warehousing.

A **Star Schema** consists of a central fact table surrounded by completely denormalized dimension tables with single-table joins. A **Snowflake Schema** normalizes dimension tables into multiple sub-dimension tables to eliminate data redundancy at the cost of requiring more complex multi-table SQL joins!

Q6. What is the difference between Min-Max Normalization and Z-Score Standardization?

Min-Max Normalization linearly scales values into a strict range $[0, 1]$, but is heavily distorted by extreme outliers. **Z-Score Standardization** transforms data to have zero mean ($\mu = 0$) and unit variance ($\sigma = 1$), which handles outliers robustly and preserves data distribution shape.